Language Modeling

Practical Example: Autocomplete in search engines (e.g., Google) or chatbots like ChatGPT generating coherent text.

Code Snippet/Mini Program: Simple LSTM-based language model for next-word prediction on toy text.

```python
import torch
import torch.nn as nn
import numpy as np

# Toy data: sentences as sequences
data = ["the cat sat on the mat", "the dog ran in the park"]
vocab = list(set(" ".join(data).split()))  # ['the', 'cat', 'sat', 'on', 'mat', 'dog', 'ran', 'in', 'park']
word_to_idx = {w: i for i, w in enumerate(vocab)}
idx_to_word = {i: w for w, i in word_to_idx.items()}

def prepare_data(texts, seq_len=3):
    X, y = [], []
    for sent in texts:
        seq = [word_to_idx[w] for w in sent.split()]
        for i in range(len(seq) - seq_len):
            X.append(seq[i:i+seq_len])
            y.append(seq[i+seq_len])
    return torch.tensor(X), torch.tensor(y)

X, y = prepare_data(data)
```

```
model = nn.LSTM(len(vocab), 50, batch_first=True)
fc = nn.Linear(50, len(vocab))
optimizer = torch.optim.Adam(list(model.parameters()) + list(fc.parameters()))

# Train loop (mini: 10 epochs)
for epoch in range(10):
    out, (h, c) = model(X)
    out = fc(h[-1])  # Use last hidden state
    loss = nn.CrossEntropyLoss()(out, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    print(f"Epoch {epoch}: Loss {loss.item():.2f}")

# Predict next word for "the cat sat"
test_seq = torch.tensor([[word_to_idx['the'], word_to_idx['cat'], word_to_idx['sat']]])
with torch.no_grad():
    out, (h, c) = model(test_seq)
    pred = torch.argmax(fc(h[-1])).item()
print(f"Predicted next word: {idx_to_word[pred]}")  # e.g., 'on'
```

Program Application: Integrate into a mobile app for predictive text (e.g., using Hugging Face Transformers for production).

2. Image Captioning:

Practical Example: Social media apps like Instagram auto-tagging photos (e.g., "A dog playing in the park").

Code Snippet/Mini Program: Simple CNN-RNN for captioning on toy data (using pre-trained CNN features).

```python
import torch
import torch.nn as nn
import numpy as np
from torchvision import models  # For pre-trained CNN

class ImageCaptioner(nn.Module):
    def __init__(self, vocab_size=100, embed_dim=256, hidden_dim=512):
        super().__init__()
        self.cnn = models.resnet18(pretrained=True)  # Encoder
        self.cnn.fc = nn.Linear(self.cnn.fc.in_features, embed_dim)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)  # Decoder
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, image, captions=None):
        img_feat = self.cnn(image)  # Shape: (batch, embed_dim)
        if captions is None:  # Inference mode
            # Dummy caption generation (simplified: one step)
            out, _ = self.lstm(img_feat.unsqueeze(1))
            return self.fc(out.squeeze(1))
        else:
            # Training: concat img_feat as first input
            lstm_in = torch.cat([img_feat.unsqueeze(1), captions[:, :-1]], dim=1)
            out, _ = self.lstm(lstm_in)
```

```
        return self.fc(out)

# Toy usage (random image/caption data)
model = ImageCaptioner(vocab_size=50)
optimizer = torch.optim.Adam(model.parameters())
dummy_image = torch.randn(1, 3, 224, 224)  # Fake image
dummy_caption = torch.randint(0, 50, (1, 5))  # Fake caption indices

# Train step
out = model(dummy_image, dummy_caption)
loss = nn.CrossEntropyLoss()(out.reshape(-1, 50), dummy_caption.reshape(-1))
optimizer.zero_grad(); loss.backward(); optimizer.step()
print(f"Loss: {loss.item():.2f}")

# Inference: Predict caption start word
with torch.no_grad():
    pred = torch.argmax(model(dummy_image)).item()
print(f"Predicted first word index: {pred}")
```

Program Application: Web app for accessibility (e.g., describe images for visually impaired using Flask + this model).

Machine Translation

Practical Example: Google Translate app converting English to French.

Code Snippet/Mini Program: Basic seq2seq for English-to-French toy translation.

```python
```

```
import torch
import torch.nn as nn

class Seq2Seq(nn.Module):
    def __init__(self, src_vocab=100, tgt_vocab=100, embed_dim=64, hidden_dim=128):
        super().__init__()
        self.encoder = nn.LSTM(embed_dim, hidden_dim)
        self.decoder = nn.LSTM(embed_dim, hidden_dim)
        self.fc = nn.Linear(hidden_dim, tgt_vocab)
        self.embed_src = nn.Embedding(src_vocab, embed_dim)
        self.embed_tgt = nn.Embedding(tgt_vocab, embed_dim)

    def forward(self, src, tgt, teacher_forcing=True):
        src_emb = self.embed_src(src)
        _, (h, c) = self.encoder(src_emb)
        tgt_emb = self.embed_tgt(tgt)
        if teacher_forcing:
            out, _ = self.decoder(tgt_emb, (h, c))
            return self.fc(out)
        else:
            # Inference (simplified)
            outputs = []
            for t in range(tgt.size(1)):
                out, (h, c) = self.decoder(tgt_emb[:, t:t+1], (h, c))
                outputs.append(self.fc(out))
            return torch.cat(outputs, dim=1)
```

```
# Toy data: src English indices, tgt French
src = torch.tensor([[1,2,3]])  # "Hello world"
tgt = torch.tensor([[4,5,0]])  # "Bonjour monde" + EOS
model = Seq2Seq()
optimizer = torch.optim.Adam(model.parameters())

out = model(src, tgt)
loss = nn.CrossEntropyLoss()(out.reshape(-1, 100), tgt.reshape(-1))
optimizer.zero_grad(); loss.backward(); optimizer.step()
print(f"Loss: {loss.item():.2f}")
```

Program Application: Real-time chat translator in apps like WhatsApp bots.

Attention Mechanism

Brief Explanation: Allows models to focus on relevant parts of input (e.g., weighted sum of hidden states). Core in Transformers: Query-Key-Value (QKV) attention.

Practical Example: In NLP, focusing on important words for sentiment analysis.

Code Snippet/Mini Program: Simple self-attention layer.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
class Attention(nn.Module):
    def __init__(self, dim=64):
        super().__init__()
        self.scale = dim ** -0.5

    def forward(self, Q, K, V):
        scores = torch.matmul(Q, K.transpose(-2, -1)) * self.scale
        attn = F.softmax(scores, dim=-1)
        return torch.matmul(attn, V)

# Toy usage
dim = 2; seq_len=2
Q = torch.tensor([[[1,0]]])  # (1,1,2)
K = V = torch.tensor([[[1,0]], [[0,1]]])  # (1,2,2)
attn_layer = Attention(dim)
output = attn_layer(Q, K, V)
print(output)  # ≈ [[[0.73, 0.27]]]
```

Program Application: Enhance search engines to rank relevant documents.

Attention over Images

Practical Example: Object detection in self-driving cars (focus on pedestrians).

Code Snippet/Mini Program: Spatial attention on image features.

```python
import torch
import torch.nn as nn
```

```
import torch.nn.functional as F

class ImageAttention(nn.Module):
    def __init__(self, channels=3, height=224, width=224):
        super().__init__()
        self.conv = nn.Conv2d(channels, 1, 1)  # Attention map generator

    def forward(self, img):
        attn_map = torch.sigmoid(self.conv(img))  # (1,1,H,W)
        attended = img * attn_map  # Element-wise multiply
        return attended

# Toy image
img = torch.randn(1, 3, 4, 4)  # Small image
model = ImageAttention(channels=3, height=4, width=4)
output = model(img)
print(f"Attention applied: Mean intensity {output.mean():.2f}")
```

Program Application: Photo editing apps (e.g., focus enhancement in Adobe Sensei).

Hierarchical Attention

Brief Explanation: Multi-level attention (e.g., word-level then sentence-level) for complex structures like documents.

Practical Example: Document summarization (focus on key sentences).

Code Snippet/Mini Program: Hierarchical attention for text.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class HierarchicalAttention(nn.Module):
    def __init__(self, word_dim=100, sent_dim=200):
        super().__init__()
        self.word_attn = nn.Linear(word_dim, 1)
        self.sent_attn = nn.Linear(sent_dim, 1)

    def forward(self, words):  # words: (num_sents, sent_len, word_dim)
        # Word-level
        word_scores = self.word_attn(words).squeeze(-1)  # (num_sents, sent_len)
        word_attn = F.softmax(word_scores, dim=1)
        sent_emb = torch.bmm(word_attn.unsqueeze(1), words).squeeze(1)  # (num_sents, word_dim) -> (num_sents, sent_dim) assume same
        # Sentence-level
        sent_scores = self.sent_attn(sent_emb).squeeze(-1)
        sent_attn = F.softmax(sent_scores, dim=0)
        doc_emb = torch.mv(sent_attn, sent_emb)
        return doc_emb

# Toy data: 2 sentences, each 2 words
words = torch.randn(2, 2, 100)
model = HierarchicalAttention()
```

```
doc_emb = model(words)
print(f"Doc embedding shape: {doc_emb.shape}")
```

Program Application: Legal document review tools (highlight key clauses).

Monte Carlo Methods

Practical Example: Risk assessment in finance (simulate stock paths).

Code Snippet/Mini Program: Monte Carlo π estimation.

```python
import numpy as np
import matplotlib.pyplot as plt

def monte_carlo_pi(n_samples=10000):
    x, y = np.random.uniform(0, 1, n_samples), np.random.uniform(0, 1, n_samples)
    inside = (x**2 + y**2 <= 1)
    pi_est = 4 * np.sum(inside) / n_samples
    return pi_est, inside, x, y

pi_est, inside, x, y = monte_carlo_pi()
print(f"Estimated π: {pi_est:.4f}")

# Plot
plt.scatter(x[inside], y[inside], c='blue', s=1)
plt.scatter(x[~inside], y[~inside], c='red', s=1)
plt.title(f'Monte Carlo π ≈ {pi_est:.4f}')
```

```
plt.show()
```

Program Application: Weather forecasting simulations.

Local Independencies in a Markov Network

Practical Example: Gene interaction networks (independence given regulators).

Code Snippet/Mini Program: Simulate Markov network with independencies.

```python
import numpy as np
from pgmpy.models import MarkovModel
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import ExactInference

# Simple chain: A-B-C
model = MarkovModel([('A', 'B'), ('B', 'C')])

# CPDs (simplified)
cpd_a = TabularCPD('A', 2, [[0.5], [0.5]])
cpd_b = TabularCPD('B', 2, [[0.4, 0.6], [0.6, 0.4]], evidence=['A'], evidence_card=[2])
cpd_c = TabularCPD('C', 2, [[0.3, 0.7], [0.7, 0.3]], evidence=['B'], evidence_card=[2])
model.add_cpds(cpd_a, cpd_b, cpd_c)
model.check_model()

infer = ExactInference(model)
```

```
# Query: P(A=0, C=0 | B=0) should factorize
prob = infer.query(variables=['A', 'C'], evidence={'B': 0})
print(prob.values)  # e.g., [0.2, 0.3, 0.3, 0.2] for joint
print("Independence: P(A,C|B=0) = P(A|B=0) * P(C|B=0)")
```

Program Application: Recommendation systems (user-item independencies given context). (Requires `pip install pgmpy`).

Joint Distributions

Practical Example: Weather prediction (joint rain/temp).

Code Snippet/Mini Program: Compute joint/marginals.

```python
import numpy as np
from scipy.stats import multivariate_normal

# 2D joint Gaussian
mean = [0, 0]
cov = [[1, 0.5], [0.5, 1]]
mvn = multivariate_normal(mean, cov)

# Sample and estimate joint P(X<0, Y<0)
samples = mvn.rvs(1000)
joint_est = np.mean((samples[:,0] < 0) & (samples[:,1] < 0))
true_joint = mvn.cdf([0,0])
print(f"Estimated joint P(X<0,Y<0): {joint_est:.2f}, True: {true_joint:.2f}")
```

Program Application: Multi-sensor IoT data analysis.

Concept of a Latent Variable

Practical Example: Topic modeling (latent topics in documents).

Numerical Example: Observed Z ~ N(μ,σ), latent H ~ N(0,1). E[log P(Z|H)] for inference.

Code Snippet/Mini Program: Simple latent variable model (linear Gaussian).

```
import torch
import torch.nn as nn
import torch.distributions as dist
import numpy as np

class LatentModel(nn.Module):
    def __init__(self, latent_dim=1, obs_dim=2):
        super().__init__()
        self.mu_latent = nn.Parameter(torch.tensor(0.))  # Prior mean for latent h
        self.sigma_latent = nn.Parameter(torch.tensor(1.))  # Prior std for latent h (positive)
        self.A = nn.Parameter(torch.randn(obs_dim, latent_dim) * 0.5)  # Linear mapping: z = A h
        self.sigma_obs = nn.Parameter(torch.tensor(0.1))  # Noise std for observed z

    def sample(self, num_samples=1):
        # Sample latent h ~ N(mu_h, sigma_h)
        latent_dist = dist.Normal(self.mu_latent, torch.abs(self.sigma_latent))  # Ensure positive sigma
```

```
        h = latent_dist.sample((num_samples, 1))

        # Sample observed z ~ N(A h, sigma_z)
        obs_mean = h @ self.A.t()  # (num_samples, obs_dim)
        obs_dist = dist.Normal(obs_mean, torch.abs(self.sigma_obs))
        z = obs_dist.rsample()  # Reparameterized sample for differentiability
        return h, z

    def log_likelihood(self, observed):
        # Marginal log P(z) approximated via integration over h (simplified: Monte Carlo)
        # For exact: could use closed-form for Gaussian, but MC for generality
        num_mc = 100
        h_mc = dist.Normal(self.mu_latent, torch.abs(self.sigma_latent)).sample((num_mc,
len(observed), 1))
        obs_mean_mc = h_mc @ self.A.t()
        log_probs = dist.Normal(obs_mean_mc,
torch.abs(self.sigma_obs)).log_prob(observed.unsqueeze(0)).sum(-1)  # Sum over
obs_dim
        log_lik = torch.logsumexp(log_probs, dim=0) - np.log(num_mc)  # MC estimate
        return log_lik.mean()

# Toy usage: Generate observed data and train the model
model = LatentModel(latent_dim=1, obs_dim=2)
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

# Toy observed data: 50 samples from true latent model (h ~ N(1, 0.5), z = [h, 2h] +
noise)
true_h = torch.normal(1.0, 0.5, (50, 1))
```

```
true_z = torch.cat([true_h, 2 * true_h], dim=1) + torch.normal(0, 0.1, (50, 2))
observed_data = true_z

# Train (20 epochs): Maximize log-likelihood
for epoch in range(20):
    optimizer.zero_grad()
    ll = model.log_likelihood(observed_data)
    loss = -ll  # Negative log-likelihood
    loss.backward()
    optimizer.step()
    print(f"Epoch {epoch}: Log-Likelihood {ll.item():.4f}, Loss {loss.item():.4f}")

# Numerical Example: Sample and inspect
with torch.no_grad():
    h_sample, z_sample = model.sample(num_samples=5)
    print("Latent samples (h):", h_sample.squeeze().numpy())
    print("Observed samples (z):", z_sample.numpy())
    print("Learned params - mu_latent:", model.mu_latent.item(), "sigma_latent:",
model.sigma_latent.item())
    print("Mapping A:", model.A.data.numpy())
    print("sigma_obs:", model.sigma_obs.item())
```

Restricted Boltzmann Machines (RBMs)

Practical Example: Recommendation systems (e.g., Netflix suggesting movies by learning user preferences from ratings).

Code Snippet/Mini Program: Train a simple binary RBM on toy data (e.g., MNIST-like patterns) using Contrastive Divergence (CD-1).

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np


class RBM(nn.Module):
    def __init__(self, visible_units=4, hidden_units=2):
        super().__init__()
        self.W = nn.Parameter(torch.randn(visible_units, hidden_units) * 0.1)
        self.v_bias = nn.Parameter(torch.zeros(visible_units))
        self.h_bias = nn.Parameter(torch.zeros(hidden_units))

    def energy(self, v, h):
        return -(v @ self.W @ h + v @ self.v_bias + h @ self.h_bias)

    def sample_h(self, v):
        logits = v @ self.W + self.h_bias
        probs = torch.sigmoid(logits)
        return probs, torch.bernoulli(probs)

    def sample_v(self, h):
        logits = h @ self.W.t() + self.v_bias
        probs = torch.sigmoid(logits)
        return probs, torch.bernoulli(probs)
```

```
    def forward(self, v):  # CD-1 training step
        h_probs, h = self.sample_h(v)
        v_recon_probs, v_recon = self.sample_v(h)
        h_recon_probs, _ = self.sample_h(v_recon)
        return v_recon_probs, h_recon_probs

# Toy data: 100 samples of 4D binary vectors (e.g., patterns)
data = torch.bernoulli(torch.tensor([[0.8,0.2,0.8,0.2]]).repeat(100, 1).float())
model = RBM(visible_units=4, hidden_units=2)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)

# Train (10 epochs)
for epoch in range(10):
    recon_loss = 0
    for batch in data.split(10):  # Mini-batches
        v_recon_probs, _ = model(batch)
        loss = F.binary_cross_entropy(v_recon_probs, batch)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        recon_loss += loss.item()
    print(f"Epoch {epoch}: Recon Loss {recon_loss/len(data):.4f}")

# Generate sample
with torch.no_grad():
    h = torch.bernoulli(torch.sigmoid(model.h_bias)).unsqueeze(0)
```

```
    v_gen, _ = model.sample_v(h)
    print(f"Generated visible: {v_gen}")
```

Program Application: Collaborative filtering in e-commerce apps (e.g., Amazon recommendations via learned features).

RBMs as Stochastic Neural Networks

Practical Example: Pre-training deep belief networks (DBNs) for image classification (e.g., initializing weights for medical imaging).

Code Snippet/Mini Program: RBM as stochastic NN for feature extraction (mean-field approximation for inference).

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class StochasticRBM(nn.Module):
    def __init__(self, visible=784, hidden=256):  # MNIST-like
        super().__init__()
        self.W = nn.Parameter(torch.randn(visible, hidden) * 0.01)
        self.v_bias = nn.Parameter(torch.zeros(visible))
        self.h_bias = nn.Parameter(torch.zeros(hidden))

    def hidden_probs(self, v):
        return torch.sigmoid(v @ self.W + self.h_bias)  # Stochastic activation
```

```
    def visible_recon(self, h_probs):
        return torch.sigmoid(h_probs @ self.W.t() + self.v_bias)

# Toy MNIST-like data (784 pixels, flattened)
data = torch.rand(10, 784) > 0.5  # Binary images
data = F.normalize(data.float(), p=1)  # Simple preprocess
model = StochasticRBM()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Train as stochastic NN (5 epochs)
for epoch in range(5):
    h_probs = model.hidden_probs(data)
    v_recon = model.visible_recon(h_probs)
    loss = F.mse_loss(v_recon, data)  # Reconstruction as NN loss
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    print(f"Epoch {epoch}: MSE {loss.item():.4f}")

# Extract stochastic features
with torch.no_grad():
    features = model.hidden_probs(data[0:1])
    print(f"Stochastic hidden features mean: {features.mean():.2f}")
```

Program Application: Anomaly detection in industrial IoT apps (e.g., stochastic modeling of sensor noise).

Unsupervised Learning with RBMs

Practical Example: Feature learning for fraud detection (unsupervised patterns in transaction data).

Code Snippet/Mini Program: Denoising RBM for unsupervised noise removal on toy images.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np

class DenoisingRBM(nn.Module):
    def __init__(self, n_visible=100, n_hidden=50):
        super().__init__()
        self.W = nn.Parameter(torch.randn(n_visible, n_hidden) * 0.1)
        self.v_bias = nn.Parameter(torch.zeros(n_visible))
        self.h_bias = nn.Parameter(torch.zeros(n_hidden))

    def sample_hidden(self, v):
        return torch.sigmoid(v @ self.W + self.h_bias), torch.bernoulli(torch.sigmoid(v @ self.W + self.h_bias))

    def sample_visible(self, h):
```

```
        return torch.sigmoid(h @ self.W.t() + self.v_bias), torch.bernoulli(torch.sigmoid(h @ self.W.t() + self.v_bias))

    def denoise(self, noisy_v):
        h_prob, h = self.sample_hidden(noisy_v)
        v_recon_prob, _ = self.sample_visible(h)
        return v_recon_prob

# Toy data: 100D vectors with noise
clean_data = torch.rand(20, 100) > 0.5
noise = torch.rand_like(clean_data) < 0.2  # 20% noise
noisy_data = torch.where(noise, 1 - clean_data, clean_data).float()

model = DenoisingRBM()
optimizer = torch.optim.SGD(model.parameters(), lr=0.05)

# Unsupervised training (10 epochs)
for epoch in range(10):
    total_loss = 0
    for noisy in noisy_data:
        v_recon = model.denoise(noisy.unsqueeze(0))
        loss = F.binary_cross_entropy(v_recon, clean_data[0].unsqueeze(0))  # Use clean for supervision in denoising
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch}: Denoising Loss {total_loss/len(noisy_data):.4f}")
```

```
# Denoise a sample
with torch.no_grad():
    denoised = model.denoise(noisy_data[0].unsqueeze(0))
    print(f"Original noisy mean: {noisy_data[0].mean():.2f}, Denoised: {denoised.mean():.2f}")
```

Program Application: Audio denoising in voice assistant apps (e.g., Siri cleaning background noise).

Setting up a Markov Chain for RBMs

Practical Example: Sampling diverse faces in generative art (e.g., This Person Does Not Exist).

Code Snippet/Mini Program: Gibbs sampling Markov chain for RBM generation.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class RBMMarkov(nn.Module):
    def __init__(self, v_size=5, h_size=3):
        super().__init__()
        self.W = nn.Parameter(torch.randn(v_size, h_size))
        self.vb = nn.Parameter(torch.zeros(v_size))
```

```
        self.hb = nn.Parameter(torch.zeros(h_size))

    def gibbs_step(self, v):
        # P(h|v)
        ph = torch.sigmoid(torch.mm(v, self.W) + self.hb)
        h = torch.bernoulli(ph)
        # P(v|h)
        pv = torch.sigmoid(torch.mm(h, self.W.t()) + self.vb)
        v_new = torch.bernoulli(pv)
        return v_new, ph

# Pre-trained toy RBM (assume trained)
model = RBMMarkov()
model.W.data = torch.tensor([[0.5, -0.2, 0.1], [0.3, 0.4, -0.1], [-0.1, 0.2, 0.6], [0.4, -0.3, 0.2], [0.1, 0.5, -0.4]])
model.vb.data = torch.zeros(5)
model.hb.data = torch.zeros(3)

# Markov chain: 10 steps from initial v
v = torch.tensor([[1., 0., 1., 0., 1.]])
chain = [v.clone()]
for _ in range(10):
    v_new, _ = model.gibbs_step(v)
    chain.append(v_new)
    v = v_new

print("Markov Chain samples:")
```

```
for i, sample in enumerate(chain):
    print(f"Step {i}: {sample}")
```

Program Application: MCMC simulation in drug discovery (sampling molecular structures).

Generative Adversarial Networks (GANs) - Architecture

Practical Example: Deepfakes for video synthesis.

Code Snippet/Mini Program: Simple GAN for 1D data generation (e.g., approximating Gaussian).

```python
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

class Generator(nn.Module):
    def __init__(self, z_dim=10, data_dim=1):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(z_dim, 32), nn.ReLU(), nn.Linear(32, data_dim), nn.Tanh())

    def forward(self, z):
        return self.net(z)
```

```
class Discriminator(nn.Module):
    def __init__(self, data_dim=1):
        super().__init__()
        self.net = nn.Sequential(nn.Linear(data_dim, 32), nn.ReLU(), nn.Linear(32, 1),
nn.Sigmoid())

    def forward(self, x):
        return self.net(x)

# Toy real data: N(0,1)
real_data = torch.randn(1000, 1) * 0.5 + 0.5  # [0,1] scaled
z_dim = 10
G = Generator(z_dim)
D = Discriminator()
opt_G = optim.Adam(G.parameters(), lr=0.001)
opt_D = optim.Adam(D.parameters(), lr=0.001)
criterion = nn.BCELoss()

# Train (50 epochs)
for epoch in range(50):
    # Train D
    real_labels = torch.ones(64, 1)
    fake_labels = torch.zeros(64, 1)
    z = torch.randn(64, z_dim)
    fake_data = G(z)
    d_real = D(real_data[:64])
```

```
    d_fake = D(fake_data.detach())
    loss_D = criterion(d_real, real_labels) + criterion(d_fake, fake_labels)
    opt_D.zero_grad(); loss_D.backward(); opt_D.step()

    # Train G
    d_fake_g = D(fake_data)
    loss_G = criterion(d_fake_g, real_labels)
    opt_G.zero_grad(); loss_G.backward(); opt_G.step()

    if epoch % 10 == 0:
        print(f"Epoch {epoch}: D Loss {loss_D.item():.4f}, G Loss {loss_G.item():.4f}")

# Generate samples
with torch.no_grad():
    z_test = torch.randn(5, z_dim)
    gens = G(z_test)
    print(f"Generated samples: {gens.squeeze()}")
```

Program Application: Synthetic data generation for privacy-preserving ML in healthcare apps.

Generative Adversarial Networks (GANs) - Applications

Practical Example: Art generation (e.g., DALL-E creating images from text) or super-resolution in satellite imagery.

Code Snippet/Mini Program: Conditional GAN (cGAN) for digit generation (toy: generate even/odd numbers).

```python
import torch
import torch.nn as nn
import torch.optim as optim

class cGenerator(nn.Module):
    def __init__(self, z_dim=10, cond_dim=1, out_dim=1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim + cond_dim, 32), nn.ReLU(),
            nn.Linear(32, out_dim), nn.Sigmoid()
        )

    def forward(self, z, cond):
        return self.net(torch.cat([z, cond], dim=1))

class cDiscriminator(nn.Module):
    def __init__(self, data_dim=1, cond_dim=1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(data_dim + cond_dim, 32), nn.ReLU(),
            nn.Linear(32, 1), nn.Sigmoid()
        )

    def forward(self, x, cond):
```

```
        return self.net(torch.cat([x, cond], dim=1))

# Toy data: even (cond=0) or odd (cond=1) numbers [0,1]
real_even = torch.rand(500, 1) * 0.5  # Low values for even
real_odd = torch.rand(500, 1) * 0.5 + 0.5  # High for odd
conds_even = torch.zeros(500, 1)
conds_odd = torch.ones(500, 1)

G = cGenerator()
D = cDiscriminator()
opt_G = optim.Adam(G.parameters(), lr=0.001)
opt_D = optim.Adam(D.parameters(), lr=0.001)
criterion = nn.BCELoss()

# Train (20 epochs, simplified)
for epoch in range(20):
    # Even class
    z = torch.randn(32, 10)
    cond = torch.zeros(32, 1)
    fake = G(z, cond)
    real_labels = torch.ones(32, 1)
    fake_labels = torch.zeros(32, 1)

    d_real = D(real_even[:32], cond)
    d_fake = D(fake.detach(), cond)
    loss_D = criterion(d_real, real_labels) + criterion(d_fake, fake_labels)
    opt_D.zero_grad(); loss_D.backward(); opt_D.step()
```

```
d_fake_g = D(fake, cond)
loss_G = criterion(d_fake_g, real_labels)
opt_G.zero_grad
```